ASSIGNMENT-11.1

Name:Spriha Gajula

Bt-13

Hlt.no:2303A51850

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack:

pass

code:

```python
class Stack:
    def __init__(self):
        self.items = []
    def push(self, item):
        self.items.append(item)
    def pop(self):
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self.items.pop()
    def peek(self):
        if self.is_empty():
            raise IndexError("Peek from an empty stack")
        return self.items[-1]
    def is_empty(self):
        return len(self.items) == 0
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print(stack.peek())  # Output: 3
    print(stack.pop())   # Output: 3
    print(stack.is_empty())  # Output: False
    print(stack.pop())   # Output: 2
    print(stack.pop())   # Output: 1
    print(stack.is_empty())  # Output: True
```

Explore and understand your code

Ask Auto

Output:

```
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py> & "C:/Users/Spriha Gajula/anaco
nda3/envs/spriha1850/python.exe" "c:/Users/Spriha Gajula/OneDrive/Desktop/ai.py/a
i.11.1.py"
3
3
False
2
1
True
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py>
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

code:

```python
class Queue:
    def __init__(self):
        self.items = []
    def enqueue(self, item):
        self.items.append(item)
    def dequeue(self):
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue")
        return self.items.pop(0)
    def peek(self):
        if self.is_empty():
            raise IndexError("Peek from an empty queue")
        return self.items[0]
    def size(self):
        return len(self.items)
    def is_empty(self):
        return len(self.items) == 0
# Example usage:
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    print(queue.peek())  # Output: 1
    print(queue.dequeue())  # Output: 1
    print(queue.size())  # Output: 2
    print(queue.dequeue())  # Output: 2
    print(queue.dequeue())  # Output: 3
    print(queue.is_empty())  # Output: True
```

Output:

```
PROBLEMS 98   OUTPUT   TERMINAL   ...   Python
2
3
i.11.1.py"
1
1
2
2
3
2
3
True
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py>
```

justification:

```
#justification:
The Stack class implements a last-in-first-out (LIFO) data structure, allowing y
to push items onto the stack, pop items off the stack, peek at the top item,
and check if the stack is empty.
The Queue class implements a first-in-first-out (FIFO) data structure, allowing
you to enqueue items at the end of the queue, dequeue items from the front
of the queue, peek at the front item, check the size of the queue, and
check if the queue is empty.
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
    def insert(self, data):#Inserts a new node with the given data at the end of
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node
    def display(self):#Displays the contents of the linked list.
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()
# Example usage:
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(1)
    linked_list.insert(2)
    linked_list.insert(3)
    linked_list.display()  # Output: 1 2 3
```

Output:

```
True
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py>
True
True
True
True
```

Justification:

```
#justification:
The Node class represents a single node in the linked list, containing data
and a reference to the next node. The LinkedList class manages the linked list,
allowing you to insert new nodes at the end of the list and display the contents
of the list. The insert method creates a new node and adds it to the end of the
list, while the display method traverses the list and prints each node's data.
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

code:

```
* BST implementation with recursive insert and traversal methods.
Here is a functional implementation of a Binary Search Tree (BST) with insert and in-order traversal methods:'''
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
class BST:
    def __init__(self):
        self.root = None
    def insert(self, data):#Inserts a new node with the given data into the BST.
        if not self.root:
            self.root = Node(data)
        else:
            self._insert_recursive(self.root, data)
    def _insert_recursive(self, node, data):
        if data < node.data:
            if node.left is None:
                node.left = Node(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = Node(data)
            else:
                self._insert_recursive(node.right, data)
    def in_order_traversal(self):#Performs an in-order traversal of the BST and returns a list of values.
        return self._in_order_recursive(self.root)
    def _in_order_recursive(self, node):
        res = []
        if node:
            res = self._in_order_recursive(node.left)
            res.append(node.data)
            res = res + self._in_order_recursive(node.right)
        return res
# Example usage:
```

Output:

```
[2, 3, 4, 5, 6, 7, 8]
s/Spriha Gajula/OneDrive/Desktop/ai.py/ai.11.1.py"
[2, 3, 4, 5, 6, 7, 8]
[2, 3, 4, 5, 6, 7, 8]
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py> & "C:/Users/Spriha Gajula/anaconda3/envs/sprihaidma/python.exe" "c:/Users/Spriha Gajula/OneDrive/Desktop/ai.py/s
s/Spriha Gajula/OneDrive/Desktop/ai.py/ai.11.1.py"
[2, 3, 4, 5, 6, 7, 8]
```

justification:

```
#justification:
The Node class represents a single node in the BST, containing data and references
to the left and right child nodes. The BST class manages the binary search tree,
allowing you to insert new nodes while maintaining the BST property and perform
an in-order traversal to retrieve the values in sorted order. The insert method
checks where to place the new node based on its value, while the in_order_traversal
method recursively visits the left subtree, the current node, and then the right
subtree to collect the values in sorted order.'''
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

code:

```
Here is a functional implementation of a hash table with basic insert, search, and delete methods, u
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]
    def _hash(self, key):
        return hash(key) % self.size
    def insert(self, key, value):#Inserts a key-value pair into the hash table.
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)  # Update existing key
                return
        self.table[index].append((key, value))  # Insert new key-value pair
    def search(self, key):#Searches for a value by its key and returns it if found.
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None  # Key not found
    def delete(self, key):#Deletes a key-value pair from the hash table.
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]  # Remove the key-value pair
                return True
        return False  # Key not found
```

output:

```
Spriha Gajula/OneDrive/Desktop/ai.py/ai.11.1.py"
Alice
Alice
30
None
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py>
```

Justification:

```
#justification:
The HashTable class implements a hash table data structure using chaining to handle collisions.
The _hash method computes the hash value of a key and maps it to an index in the table.
The insert method adds a key-value pair to the appropriate index, updating the value if the key
already exists. The search method looks for a key in the corresponding index and returns its value
if found, while the delete method removes a key-value pair from the table if it exists. This
implementation allows for efficient insertion, searching, and deletion of key-value pairs while
managing collisions effectively.'''
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

code:

```
class Graph:
    def __init__(self):
        self.graph = {}
    def add_vertex(self, vertex):#Adds a vertex to the graph.
        if vertex not in self.graph:
            self.graph[vertex] = []
    def add_edge(self, vertex1, vertex2):#Adds an edge between two vertices in the graph.
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)  # For undirected graph
    def display(self):#Displays the graph's adjacency list representation.
        for vertex, edges in self.graph.items():
            print(f"{vertex}: {', '.join(edges)}")
# Example usage:
if __name__ == "__main__":
    graph = Graph()
    graph.add_vertex("A")
    graph.add_vertex("B")
    graph.add_vertex("C")
    graph.add_edge("A", "B")
    graph.add_edge("A", "C")
    graph.add_edge("B", "C")
    graph.display()
    # Output:
    # A: B, C
    # B: A, C
    # C: A, B
```

Output:

```
PROBLEMS 98   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Python

PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py> & "C:/Users/Spriha Gajula/anaconda3/envs/spriha1850/python.exe" "c:/User
s/Spriha Gajula/OneDrive/Desktop/ai.py/ai.11.1.py"
A: B, C
B: A, C
C: A, B
C: A, B
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py>
```

Justification:

```
The Graph class uses a dictionary to represent the graph as an adjacency list,
where each key is a vertex and the corresponding value is a list of adjacent vertices.
The add_vertex method adds a new vertex to the graph, while the add_edge method creates an
undirected edge between two vertices by adding each vertex to the other's adjacency list. '
'The display method iterates through the graph and prints each vertex along with its connected '
'vertices, providing a clear representation of the graph's structure. This implementation allows
for efficient management of vertices and edges while maintaining a simple and intuitive interface
for graph operations.'''
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

class PriorityQueue:

pass

code:

```
import heapq
class PriorityQueue:
    def __init__(self):
        self.elements = []
    def enqueue(self, item, priority):#Adds an item to the priority queue with a given priority.
        heapq.heappush(self.elements, (priority, item))
    def dequeue(self):#Removes and returns the item with the highest priority (lowest priority value)
        if not self.is_empty():
            return heapq.heappop(self.elements)[1]
        raise IndexError("Dequeue from an empty priority queue")
    def display(self):#Displays the contents of the priority queue.
        print("Priority Queue Contents:")
        for priority, item in sorted(self.elements):
            print(f"Priority: {priority}, Item: {item}")
    def is_empty(self):
        return len(self.elements) == 0
# Example usage:
```

Output:

```
PROBLEMS 10   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                    Python
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py> & "C:/Users/Spriha Gajula/anaconda3/envs/sprihal650/python.exe" "c:/User
Priority: 2, Item: Task A
Priority: 3, Item: Task C
Task B
Task A
Task C
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py>
```

Justification:

```
#justification:
The PriorityQueue class uses the heapq module to maintain a min-heap based on priority values
The enqueue method adds items to the priority queue with their associated priorities, while
the dequeue method removes and returns the item with the highest priority (lowest priority value).
 The display method provides a clear view of the contents of the priority queue, showing each item
 along with its priority. The is_empty method checks if the priority queue is empty before attempting
 to dequeue an item, ensuring that operations are performed safely. This implementation allows for
 efficient management of tasks based on their priorities while providing a straightforward interface
 for adding, removing, and viewing items in the priority queue.'''
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using

collections.deque.

Sample Input Code:

class DequeDS:

pass

code:

```
from collections import deque
from os import remove
class DequeDS:
    def __init__(self):
        self.deque = deque()
    def insert_front(self, item):#Inserts an item at the front of the deque.
        self.deque.appendleft(item)
    def insert_rear(self, item):#Inserts an item at the rear of the deque.
        self.deque.append(item)
    def remove_front(self):#Removes and returns an item from the front of the deque.
        if not self.is_empty():
            return self.deque.popleft()
        raise IndexError("Remove from an empty deque")
    def remove_rear(self):#Removes and returns an item from the rear of the deque.
        if not self.is_empty():
            return self.deque.pop()
        raise IndexError("Remove from an empty deque")
    def is_empty(self):
        return len(self.deque) == 0
    def display(self):
        print("Deque Contents:", list(self.deque))
```

Output:

```
PROBLEMS 215   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                    Python
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py> & "C:/Users/Sprina Gajula/anaconda3/envs/sprina1858/python.exe" "c:/User
Priority: 2, Item: Task A
Priority: 3, Item: Task C
Task B
Task A
Task C
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py>
```

Justification:

```
#justification:
The DequeDS class utilizes the collections.deque module to implement a double-ended queue,
 allowing for efficient insertion and removal of items from both ends. The insert_front and
 insert_rear methods add items to the front and rear of the deque, respectively, while the
 remove_front and remove_rear methods remove and return items from the front and rear, ensuring
that operations are performed safely by checking if the deque is empty before attempting to remove
items. The display method provides a clear view of the current contents of the deque, making it
easy to visualize the state of the data structure. This implementation allows for flexible
management of items in a double-ended queue while maintaining a simple and intuitive interface
for users.'''
```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

```python

import heapq
from datetime import datetime

class LibrarySystem:
    def __init__(self):
        self.borrowed_books = []
    def borrow_book(self, book_title, student_name, due_date_str):
        due_date = datetime.strptime(due_date_str, "%Y-%m-%d")
        heapq.heappush(self.borrowed_books, (due_date, book_title, student_name))
        print(f"Book '{book_title}' borrowed by {student_name}, due on {due_date_str}")
    def return_next_due_book(self):
        if not self.borrowed_books:
            print("No borrowed books.")
            return
        due_date, book_title, student_name = heapq.heappop(self.borrowed_books)
        print(f"Next due book returned: '{book_title}' by {student_name}, was due on {due_date.date()
    def show_all_books(self):
        if not self.borrowed_books:
            print("No borrowed books.")
            return
        print("\nBorrowed Books (sorted by due date):")
        for due_date, book_title, student_name in sorted(self.borrowed_books):
            print(f"{book_title} | Borrower: {student_name} | Due: {due_date.date()}")
# Example Usage
```

Output:

```
PROBLEMS 98   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py> & "C:/Users/Spriha Gajula/anaconda3/envs/sprihal850/pytho
Next due book returned: 'Operating Systems' by Rahul, was due on 2026-03-05

Borrowed Books (sorted by due date):
Data Structures | Borrower: Anjali | Due: 2026-03-10
Machine Learning | Borrower: Sneha | Due: 2026-03-15
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py>
```

Justification:

```
justification:
The Library Book Borrowing System is implemented using a Priority Queue (min-heap) because it
allows us to efficiently manage and serve books based on their due dates. By using a priority queue,
we can ensure that the book with the earliest due date is always returned first, which is crucial
for maintaining an organized borrowing system. This data structure provides efficient insertion and
retrieval of books based on their priority (due date), making it an ideal choice for this
feature of the Campus Resource Management System.
```

Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python
class OrderProcessingSystem:
    def __init__(self):
        self.orders = deque()
    def place_order(self, order_id, customer_name, product_name):
        order = {
            "order_id": order_id,
            "customer_name": customer_name,
            "product_name": product_name
        }
        self.orders.append(order)
        print(f"Order placed: {order}")
    def process_order(self):
        if not self.orders:
            print("No pending orders.")
            return
        order = self.orders.popleft()
        print(f"Processing order: {order}")
    def display_pending_orders(self):
        if not self.orders:
            print("No pending orders.")
            return
        print("\nPending Orders:")
        for order in self.orders:
            print(order)
```

Output:

```
PROBLEMS 98   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py> & "C:/Users/Spriha Gajula/anaconda3/envs/spriha1850/pyth
n.exe" "c:/Users/Spriha Gajula/OneDrive/Desktop/ai.py/ai.11.1.py"

Pending Orders:
{'order_id': 103, 'customer_name': 'Rahul', 'product_name': 'Headphones'}
PS C:\Users\Spriha Gajula\OneDrive\Desktop\ai.py>
```

Justification:

```
justification:
The Smart E-Commerce Order Processing System is implemented using a Queue (FIFO principle) because
it ensures that orders are processed in the exact order they were placed, maintaining fairness and
customer satisfaction. A queue allows for efficient addition of new orders at the rear and processing
of orders from the front, which is essential for managing a high volume of orders in an
e-commerce environment. This data structure provides a straightforward way to handle the flow of
orders while ensuring that no order is skipped or processed out of sequence.
```